```
        vector<vector<int>> ans;
        auto comp = [&](const vector<int>& i, const vector<int>& j){
            if(i[0]==j[0])
                return i[1] > j[1];
            return i[0] < j[0];
        };
        sort(intervals.begin(), intervals.end(), comp);
        ans.push_back(intervals[0]);
        int k =0;
        for(int i =1; i< intervals.size(); ++i){
            if( (intervals[i][0]>= ans.back()[0]) and (intervals[i][1] <= ans.back()[1]))
            {
               // Scenario 1: Completely covered suppose existing interval in vector is [1, 8] and this ith interval is [2, 6] , this is completely inside i.e
               // lies inside , hence this interval is surely removed, count this.
                ++k;
            }
            else if(intervals[i][0] > ans.back()[1]){
            // Scenario 2: Suppose vector has [1, 8]  and ith interval is  [9, 10] , totally unrelated, so push this interval in vector as this is going to t
                ans.push_back(intervals[i]);
            }
            else if ((intervals[i][0] > ans.back()[0]) and (intervals[i][0] <= ans.back()[1]) )
                // Scenario 3: vector has [1, 8] and ith interval is  [5, 10], so take maximum of both end point.
                ans.back()[1] = max(ans.back()[1], intervals[i][1]);
        }
        return intervals.size()- k;
    }
};
```

1353. Maximum Number of Events That Can Be Attended [**Medium**]

Add detailed comments in code , please check , here the key idea is we are sweeping the day as a number line, it is given in input and we mark the events on that day line.

```
class Solution {
public:
    int maxEvents(vector<vector<int>>& events) {
        sort(events.begin(), events.end());
        multiset<int> eventEndTime;
        int i =0;
        int ans = 0;
        int n = events.size();
        for(int d =1; d <=100000; ++d){
            //Delete expired event , example
            // Suppose 3 events are there  [1, 2] [1, 2] [1, 2]
            // at day 1 : attend 1st event and at day 2 attend 2md event , at day 3 , 3rd event is already expired, hence we need this kind of loop of loop tc
            while(!eventEndTime.empty() and *eventEndTime.begin() < d){
                eventEndTime.erase(eventEndTime.begin());
            }

            // put all candidate events whose start day is past the current day.

            //insert events if they can be start
            while(i < n and events[i][0] <=d){
                eventEndTime.insert(events[i][1]);
                i++;
            }

            // we can attend 1 event on 1 day , thats why if condition not while
            // adn we attend earliest ending event first , suppose we have [1, 2] & [1, 3]
            // and we are on day=2, we should attend [1,2] first otherwise at d=3 this would be expired
            if(!eventEndTime.empty() and *eventEndTime.begin()>=d){
                ++ans;
                eventEndTime.erase(eventEndTime.begin());
            }
        }
        return ans;
    }
};
```

56. Merge Intervals [**Medium**]

Using Map counter

```
        class Solution {
public:
    vector<vector<int>> merge(vector<vector<int>>& intervals) {
        map<int, int> line;
        for(auto& i : intervals){
            ++line[i[0]];
            --line[i[1]];
        }

        int count = 0;
        vector<vector<int>> ans;
        int start = 0;
        for(auto& i : line){
            // that means its a new start, store the start
            if(count ==0){
                start = i.first;
            }
            count += i.second;
            // this mean interval ends, and we can push this as answer
            if(count==0){
                ans.push_back({start, i.first});
            }
        }
        return ans;
    }
};
```

Using prev interval

```
    sort(intervals.begin(), intervals.end());// Sort y there start time
        vector<vector<int>> results;
        results.push_back(intervals[0]);
        for(int i =1; i < intervals.size(); ++i){
            if(intervals[i][0] > results.back()[1]) // a new begining
            {
                results.push_back(intervals[i]);
            }
            else
                results.back()[1] = max(results.back()[1], intervals[i][1]);
        }
        return results;
```

1589. Maximum Sum Obtained of Any Permutation [**Medium**]
1943. Describe the Painting [**Medium**]
1674. Minimum Moves to Make Array Complementary [**Medium**]

1D Hard problem

These 1D Hard problem require scanning line for each input which can lead to O(n^2) algorithm.
We have to do something special to optimize it.
Lets see with an example.

2158. Amount of New Area Painted Each Day [**Hard**]
In this problem each day we paint some section of a line.
Brute force way is scan line for each input index.

Thanks to @cjcoax and @votrubac, I am putting both line sweep version and map interval method here.
Similar problem
2251. Number of Flowers in Full Bloom [**Hard**]

Using Line Sweep

```
        auto cmp =[&](const vector<int>& a, const vector<int>& b){
            return a[1]  < b[1];
        };
        int maxEnd = (*max_element(paint.begin(), paint.end(), cmp))[1];
        int n = paint.size();
        vector<int> ans(n, 0);
        vector<vector<pair<int, int>>> line(1 + maxEnd);
        // We are marking on line that co-oridnate is
    // painted/not-painted(true-false) between which date
        for(int i =0; i < n ; ++i){
            line[paint[i][0]].push_back(make_pair(i, 1));
            line[paint[i][1]].push_back(make_pair(i, 0));
        }

        set<int> days;
        //Scan the line
        for(int i =0; i < maxEnd; ++i){

            // Who all present on this x co-ordinate?
            for(auto& [day, state] : line[i]){

                if(state)
                    days.insert(day);
                else
                    days.erase(day);
            }
            //Only the first guy can paint this line
            if(!days.empty())
                ans[*(days.begin())]++;
        }
        return ans;
```

Using Vector Interval

```
    map<int, int> m;
    vector<int> res;
    for (auto &p : pt) {
        int l = p[0], r = p[1];
        auto next = m.upper_bound(l), cur = next;

        // Step 1: suppose we have painted [1, 4] [ 5, 8] [10, 20]
        // Now a new interval [4, 21] comes upper_bound gives [5,8]
        // l = 4 then
        if (cur != begin(m) && prev(cur)->second >= l) {
            cur = prev(cur);
            l = cur->second;
        }
        else
            cur = m.insert({l, r}).first;
        int paint = r - l;
        // Next since r= 21 and that is  > 5, that means this paint is going
        // find how much shd we subtract :
        // get min of (21, 8) =  8 and then subtratc with start 5 = 3
        // now r shd be max of (21, 8) =21 , check further more intervals ca
        // in the end set curr->second = max(curr->second, r);
        while (next != end(m) && next->first < r) {
            paint -= min(r, next->second) - next->first;
            r = max(r, next->second);
            m.erase(next++);
        }
        cur->second = max(cur->second, r);
        res.push_back(max(0, paint));
    }
    return res;
```

1326. Minimum Number of Taps to Open to Water a Garden [**Hard**]
Key idea is at every point, find the maximum range.
Now sweep the line from 1 to n
if i > curr_max : not possible -1
else if i ==curr_max; current selected tap capacity is over, now we have select the next best tap which we have found in past. set that
else our current tap is good , no need to open any new tap, but keep recording next_best_tap.

```
class Solution {
public:
    int minTaps(int n, vector<int>& ranges) {
        // Find what is the max we can reach if we start opening the tap at every ith location
        vector<int> line (1+n, 0);
        for(int i =0; i <=n; ++i){
            int left = max(0, i - ranges[i]);
            int right = min(n, i + ranges[i]);
            line[left] = max(line[left], right);
        }
        // Sweep line
        // Lets 0th tap as best one
        int curr = line[0];
        int next_best = 0;
        int ans = 1;
        for(int i = 1; (i <=n) and (curr < n); ++i){
            // we cannot reach to this ith that means not possible at all !
            if( i > curr)
                return -1;
            else if ( i == curr){
                // curr reach its end , time to select next best
                next_best = max(next_best, line[i]);
                ++ans;
                curr = next_best; // assign next_best to curr
                next_best =0;//rest next_best as 0
            }
            else{
                // we still are in range of curr, no need to open a new tap but keep checking what next best tap of highest range we can open next.
                next_best = max(next_best, line[i]);
            }
        }
        return ans;;
    }
};
```

732. My Calendar III [**Hard**]
Same as Meeting Room II as explained earlier.

759. Employee Free Time [**Hard**]
Same conepts, mark start and end time on line, when will everyone is free i.e. when count is 0.
So whenever count is 0 , it mark the begining of an interval, also set a flag , so that after we non-zero from 0, we use that as closing interval.

```
    map<int, int> line;
        for(auto& s : schedule){
            for(auto& i :  s){
                ++line[i.start];
                --line[i.end];
            }
        }

        int count = 0;
        bool found = false;
        vector<Interval> ans;
        for(auto& x : line){
            count += x.second;
            if(found){
                ans.back().end = x.first;
                found = false;
            }
            if(count == 0){
                // mark begining of new interval
                ans.push_back(Interval(x.first, -1));
                found = true;
            }
        }
        ans.pop_back();
        return ans;
```

1851. Minimum Interval to Include Each Query [**Hard**]
We use multiset to track size of interval, multiset.begin() will always gives you smallest size interval which is still active.
Sweep the line,
If it is start of interval, insert the size in multiset,
If it is end of interval remove from multi set.
If there is a query here , just add the first (which is smallest size) to answer index

```
        map<int, set<pair<int, int>>> line;
        // -1 :  Entry  1 : Exit , 2 : Query & size of interval
        for(auto& i : intervals){
            int size = i[1] -i[0] + 1;
            line[i[0]].insert(make_pair(-1, size));
            line[i[1]+1].insert(make_pair(1, size));
        }

        for(int i =0; i < queries.size(); ++i){
            line[queries[i]].insert(make_pair(2, i));
        }
        vector<int> ans(queries.size(), -1);
        multiset<int> sizes;
        for(auto& [x, intervals] :  line){
            for(auto& i : intervals){
                if(i.first==-1)
                    sizes.insert(i.second);
                else if (i.first==1)
                    sizes.erase(sizes.lower_bound(i.second));
                else if (i.first ==2 and !sizes.empty())
                    ans[i.second] = *(sizes.begin());

            }

        }
        return ans;
```

2D Problem's

2D problems are slightly tricky as there extra dimension have to be tracked.
Lets see this with an example.